

Stat 5810/6810 — Homework 4 — Tu 10/16/2012

General Instructions

Due: Fr 10/26/2012, 11:59pm (by e-mail)

When you use a search engine to look for a web page, the search engine looks through its cache.

The cache is created by regularly crawling the web and taking copies of every page that has changed since the last time the search engine visited the site. It is time consuming and costly to crawl the web. In this assignment, you will analyze data collected to help answer the question: How often do you need to crawl the web in order to keep its stock fresh for the search engine?

The data collected are measurements on 500 web sites that were visited regularly over a period of time to see whether or not they have changed. We will study the properties of the changes. These data are in a list of 500 elements, one for each web site. Each element is an integer vector that records whether the web site has changed since the previous visit. For example,

```
> Cache500[[8]]  
[1] 3 23 30 32 39 42 45 62 78 82 147 151 155 175
```

indicates that the 8th element of `Cache500` first changed on the third visit; it changed a total of fourteen times, and the final change observed was at the 175th visit to the site.

Ultimately, we want to see whether the Poisson model for random scatter describes the times at which a change was observed for our web sites. If this is the case, the times should be uniformly distributed, like the x coordinates of raindrops on a square of concrete. We will look at a subset of all the web sites (described in the .R file) and conduct analysis of these sites.

If the Poisson scatter model is true, the times of changes should be scattered fairly uniformly on the interval from 0 to 720 (the interval of observation). Rather than making a histogram, it can be more informative to examine a quantile plot where we compare the quantiles of the distribution of observed changes to the uniform quantiles. The points should roughly fall on a straight line (for some sites it will be closer than for others).

To help decide whether the Poisson model is appropriate, we can simulate data according to a random uniform, make a quantile plot, and see how close the quantile plot for the simulated data is to a line. The .R file will take you through the calculation of a statistic used to measure the closeness to a line, the maximum difference between the observed value and the expected value (on the line). By comparing the empirical distribution of the simulated values of this statistic to the real value from a data vector, we can more

formally assess whether the quantile plot is as close to a line as what we would really get from uniformly generated data.

Below are detailed instructions for tasks 6 & 7 from the .R file:

- 6) Write a function called `SimMaxDiff` that takes as one argument a vector (for example a data vector like `Cache500[[8]]`) and another argument `n` which has a default value of 1000. The function should do the following:
 1. Create a matrix of random uniform data between 0 to 720, with number of rows equal to the length of the data vector and number of columns equal to `n`. Essentially each column is what you would expect if the original data come from the uniform distribution on $(0, 720)$.
 2. Use `apply()` and the function `MaxDiff()` to get a vector of length `n` which gives the maximum differences for each column of this simulated data set. Notice that your random data are not sorted from lowest to highest the way that the `Cache500` data are, this is why we needed to use the sort function in the first line of the `MaxDiff()` function.
 3. Use the `rank()` function to determine where the result from `MaxDiff()` for the real data ranks amongst the simulated columns. The answer to this step should be a number between 1 (if the result from `MaxDiff()` for this website is smaller than all the simulated values) and `n+1` (if this website has a bigger value from `MaxDiff()` than all the simulated values. Note that every time you run this function you may get a (slightly) different rank.
 4. Use `return()` to pass back the value you just found from 1 to `n+1`.
- 7) Write another function called `HistRanks` that uses the previous two functions and `sapply()` to get a rank for each website in `whichOK` amongst simulated data. (The input to this function should be a subset of `Cache500` containing the data from the `whichOK` sites, i.e. `Cache500[whichOK]`.) The rank for each Web site should be between 1 and `n+1`, for now use the default value of `n = 1000`.
 1. Scale the ranks between 0 and 1 (in other words, divide by `n+1`).
 2. Make a histogram of the scaled ranks for all the websites in `whichOK`. Include on your histogram a vertical red line through 0.95. Use the `breaks` argument to make it so that the edge of a bar in the histogram is at 0.95.
 3. Use `return()` to pass back the vector of scaled ranks.